EduOS Simulator Report

Operating Systems Assignment

1. Introduction

This project implements an educational operating system simulator called EduOS using C and Python. The simulator demonstrates important operating system concepts including process management, thread synchronization, deadlocks, CPU scheduling, and inter-process communication. The goal of the project was to provide practical understanding of how operating systems manage resources, processes, and threads.

2. Objectives

- Simulate process management using PCB structures
- Demonstrate multithreading and synchronization
- Implement CPU scheduling algorithms
- Demonstrate deadlock and IPC concepts
- Integrate C and Python modules
- Visualize scheduling using Gantt charts

3. System Design

The project was divided into two major components. The C module handled low-level operating system simulation including process management, threading, synchronization, and deadlock demonstrations. The Python module implemented CPU scheduling algorithms and visualizations.

4. Process Control Block (PCB)

A Process Control Block (PCB) structure was implemented to store process information. The PCB contained important fields such as process ID (PID), process state, priority, burst time, arrival time, memory requirements, and thread count. The PCB table acted as the simulated operating system process table.

5. Process Management

The EduOS simulator implemented simplified versions of the following operating system functions:

• edu_fork() – simulated process creation
• edu_exec() – simulated loading a program into a process
• edu_exit() – simulated process termination
• edu_ps() – displayed the process table

The process table was serialized into JSON format to support integration with Python modules.

6. Threading and Synchronization

Multithreading was implemented using pthreads. A shared counter variable was intentionally accessed by multiple threads simultaneously to demonstrate race conditions. Without synchronization, inconsistent counter results were produced. Mutex locks were later introduced to protect the critical section and ensure data consistency.

7. Deadlock Demonstration

Deadlock was demonstrated using two mutex locks acquired in opposite order by two threads. This caused circular waiting, where each thread waited indefinitely for a resource held by the other thread. The demonstration helped illustrate the four conditions necessary for deadlock:

- Mutual exclusion
- Hold and wait
- No preemption
- Circular wait

8. Inter-Process Communication (IPC)

The project explored inter-process communication concepts including pipes and shared memory. Pipes allow processes to communicate sequentially using data streams, while shared memory allows multiple processes to access the same memory region for faster communication.

9. CPU Scheduling Algorithms

Several CPU scheduling algorithms were implemented in Python:

- First Come First Serve (FCFS)
- Shortest Job First (SJF)
- Priority Scheduling
- Round Robin Scheduling

Each algorithm was evaluated using waiting time and turnaround time metrics. Gantt charts were generated using matplotlib to visualize process execution order.

10. Integration Between C and Python

The C simulator generated PCB information in JSON format. Python modules later loaded this JSON data to simulate scheduling algorithms. A controller module was created to automatically execute the C simulator and load generated process data.

11. Challenges Faced

Several challenges were encountered during development:

- Understanding thread synchronization concepts
- Handling race conditions and mutex locks
- Windows compatibility issues with POSIX functions
- Integrating multiple programming languages
- Managing multi-file C projects

12. Conclusion

The EduOS simulator successfully demonstrated key operating system concepts including process management, threading, synchronization, deadlocks, IPC, and CPU scheduling. The project provided practical experience in systems programming and improved understanding of how operating systems coordinate processes and system resources.

References

1. Silberschatz, A., Galvin, P.B., & Gagne, G. Operating System Concepts.
2. Python Documentation.
3. GCC Compiler Documentation.
4. pthreads Documentation.